

PERSONAL STATEMENT

Jianming Xing

City University of Hong Kong
MSc in Computer and Information Engineering

A learned robot policy is never only a model. I came to understand that its behavior depends on the semantics and integrity of the surrounding system. Timestamps must align observations with actions, while state and action fields must retain the same meaning from collection to execution. Datasets need structural and motion validation; a server must return the expected chunks; inference must occur on a meaningful robot state. These dependencies shifted my focus from isolated AI prediction to reliable learning systems for embodied agents.

To gain end-to-end competence across machines and computation, I built my education as a bridge rather than a replacement. I completed a B.Eng. in Mechanical Design, Manufacturing and Automation at Harbin Institute of Technology, Weihai, in June 2025. That training grounded me in mechanism behavior, kinematics, and physical constraints. I am now pursuing a second bachelor's degree in Computer Science and Technology at Harbin Institute of Technology, Shenzhen, expected in June 2027. This expansion lets me reason both about a machine's physical capabilities and about how data structures, algorithms, interfaces, and runtime decisions determine its behavior. Together, these disciplines help me trace a learned capability from representation to physical execution.

My research asks how action representation shapes what a vision-language-action model can learn. As a Research Assistant at iLearn, HIT Shenzhen, I am second author of Hermite Action Tokenizer for VLA, an unsubmitted working paper, and I am responsible for the discrete autoregressive tokenization branch. I worked on splitting seven-dimensional action chunks into six-dimensional end-effector trajectories and gripper states, using shared breakpoints to remove redundant endpoints and quantization seams while coding gripper changes separately. In this branch, I also compared adaptive MLP-based and fixed Hermite segmentation. This made evaluation design part of my research: I needed to distinguish evidence about an action encoding from evidence about a complete policy. In an offline codec benchmark averaging four LIBERO subsets with 8-by-7 action chunks, Hermite's Endpoint MSE was approximately 99.7% lower than FAST and 82.6% lower than BEAST. By isolating the codec from a complete policy loop, this benchmark tested representation fidelity under a defined protocol, while leaving rollout success and physical-robot reliability as open system-level questions.

Complementary systems work on a Dobot CR5 addressed questions beyond an offline codec benchmark. During deployment testing, non-blocking execution allowed new policy inference to use an intermediate in-motion state before the preceding action chunk finished, producing back-and-forth corrections. I therefore used blocking short-horizon ServoP action chunks, accepting lower apparent frequency to preserve state-action consistency. This was a serving and execution-semantics decision, not a change in model score. To make the rest of the loop equally explicit, I anchored an HDF5 synchronization pipeline to RGB timestamps and aligned robot states, target actions, and gripper feedback through binary search and linear interpolation. I applied schema, timing, motion, and workspace checks before downstream use, then converted synchronized episodes into LeRobot v2.0 while preserving seven-dimensional robot-plus-gripper state and action semantics. I adapted OpenPI with a training configuration, WebSocket policy serving, and a CR5 inference client that assembled observations, received action chunks, and executed robot targets. Hermite and CR5 were separate efforts, but together they clarified complementary responsibilities. Algorithmic evaluation tests a representation under a bounded protocol; systems reliability depends on clocks, schemas, service contracts, observations,

and execution semantics remaining coherent throughout the loop.

Supporting projects reinforced this discipline. For a weld-seam HMI, I integrated C++/Qt controls with Python/PyTorch segmentation inference through QProcess and OpenGL visualization. I preserved boundaries between the desktop interface, inference process, and robot representation instead of collapsing them into one opaque application. During the Xbotics Embodied AI Community Internship, I migrated an ANYmal-C navigation task from Isaac Lab into a MotrixLab NumPy environment. Treating the port as an interface-preservation problem, I refactored reset and step flows and added rollout and reward/termination regression checks. Across both projects, explicit contracts and tests became my tools for cross-component change.

My immediate goal is AI/ML or learning-systems R&D for embodied agents, building methods that remain dependable when connected to real data and hardware. I want graduate training to deepen my foundations in machine learning, data systems, and scalable software while testing them against sensing and control constraints. After gaining stronger real-system experience, I may consider doctoral study as an option rather than a predetermined step. I seek curricular depth and applied opportunities to turn this perspective into rigorous, transferable engineering. On the CR5, I aligned observations to RGB timestamps and carried policy chunks through WebSocket serving to robot execution. In the HERO RoboMaster team, I worked on ROS2 messaging, host-MCU communication, and time synchronization. These experiences taught me to diagnose individual interfaces, but I still need a cross-layer framework spanning sensor input, data movement, learned interpretation, cloud deployment, and computing hardware. CityUHK's MSc in Computer and Information Engineering maps directly onto that integration gap.

With the annual availability of all three technical electives still to be confirmed, EE5811 Topics in Computer Vision and EE5438 Applied Deep Learning would turn sensor data into learned interpretation. EE5608 Modern Cloud Architecture and Deployment Practices, if available and selected as one core choice, would carry model outputs across service boundaries; EE6625 Hardware Architectures for Artificial Intelligence and Machine Learning would then expose the computing substrate beneath inference. I would trace one synchronized observation as it enters a cloud-deployed vision and deep-learning service, make the hardware assumptions governing inference explicit, and follow the resulting command back to robot execution. That end-to-end trace would help me locate cross-layer failures before they are absorbed into overall model behavior in embodied-systems R&D.